
Portefeuille Assurance Polices & Sinistres

Un jeu de données volontairement imparfait pour pratiquer
le nettoyage, les jointures et les agrégations avec PySpark

Fichiers	<code>policies.csv</code> , <code>claims.csv</code>
Volumétrie	~225 polices, ~486 sinistres
Niveaux	3 niveaux de difficulté progressifs
Stack	PySpark (DataFrame API)

Table des matières

1	Contexte	2
1.1	Fichiers fournis	2
1.2	Consignes générales	2
2	Niveau 1 — Chargement et nettoyage	3
3	Niveau 2 — Jointures et agrégations	4
4	Niveau 3 — Fenêtrage et analyse avancée	5
5	Pour aller plus loin	6

1 Contexte

Vous travaillez sur le portefeuille d'un assureur fictif. Deux fichiers vous sont fournis : les **polices** en portefeuille et les **sinistres** déclarés sur ces polices.

Ce dataset contient des imperfections réalistes typiques de données de production (valeurs manquantes, doublons, formats hétérogènes, valeurs aberrantes, références orphelines). Elles ne sont **pas** listées ici : les identifier fait partie de l'exercice, au fil des questions.

1.1 Fichiers fournis

polices.csv

policy_id	identifiant de la police
client_id	identifiant du client
country	pays (FR, BE, CH, LU)
product	produit (Auto, Habitation, Santé, Prévoyance)
premium_amount	prime annuelle
currency	devise
start_date	date d'effet de la police
status	Active / Résiliée / Suspendue

claims.csv

claim_id	identifiant du sinistre
policy_id	référence vers la police concernée
claim_date	date de déclaration
claim_type	nature du sinistre
claim_amount	montant déclaré
status	Ouvert / Clos / En cours d'expertise

1.2 Consignes générales

- Utiliser l'API DataFrame de PySpark (pas de SQL brut, sauf mention contraire) — l'objectif est de pratiquer les transformations.
- Justifier chaque décision de nettoyage (pourquoi filtrer, pourquoi imputer, pourquoi supprimer) — ce sont ces choix qui comptent en entretien, pas seulement le résultat.
- Documenter les hypothèses prises lorsqu'une donnée est ambiguë.

2 Niveau 1 — Chargement et nettoyage

Niveau 1 — Prise en main & qualité des données

1. Charger les deux fichiers avec un schéma explicite (pas d'inférence automatique). Afficher le schéma et le nombre de lignes de chacun.
2. Identifier et quantifier les valeurs manquantes par colonne, pour les deux DataFrames.
3. Identifier les doublons exacts dans **policies** et les supprimer proprement.
4. Identifier les **claim_id** présents plusieurs fois avec des valeurs différentes. Proposer et justifier une règle de déduplication (ex. garder la ligne la plus récente / la plus complète).
5. **start_date** et **claim_date** contiennent des formats de date hétérogènes. Écrire une transformation qui les uniformise en type **date**.
6. Détecter les **policy_id** présents dans **claims** mais absents de **policies**. Combien de lignes cela représente-t-il ? Que décidez-vous d'en faire, et pourquoi ?
7. Détecter les montants de prime et de sinistre incohérents (négatifs, nuls, ou aberrants par rapport à la distribution générale). Justifier le seuil retenu pour qualifier un montant d'aberrant.

3 Niveau 2 — Jointures et agrégations

Niveau 2 — Croiser polices et sinistres

1. Joindre **polices** et **claims** pour obtenir, pour chaque sinistre, le pays et le produit de la police associée. Choisir le type de jointure adapté et justifier ce choix.
2. Calculer, par pays, le nombre de sinistres et le montant total des sinistres.
3. Calculer, par produit, la **fréquence de sinistralité** (nombre de sinistres / nombre de polices actives du produit).
4. Calculer un **ratio sinistres/primes** par pays et par produit (montant total des sinistres / montant total des primes). Quels sont les couples pays × produit les plus déséquilibrés ?
5. Pour chaque client (**client_id**), calculer le nombre de polices détenues et le montant total des sinistres sur l'ensemble de ses polices.
6. Identifier les 10 polices ayant généré le plus de sinistres en montant cumulé.
7. Calculer la répartition des sinistres par statut (**Ouvert** / **Clos** / **En cours d'expertise**), globalement puis par pays.

4 Niveau 3 — Fenêtrage et analyse avancée

Niveau 3 — Window functions & restitution

1. Pour chaque police, classer ses sinistres par montant décroissant à l'aide d'une fonction de fenêtrage (**rank** ou **row_number**), et ne conserver que le sinistre le plus coûteux par police.
2. Calculer, par pays et par mois, le montant cumulé des sinistres dans le temps (somme courante / *running total*).
3. Construire une table pivot croisant **country** (lignes) et **product** (colonnes), avec le montant total des sinistres en valeur.
4. Calculer l'ancienneté de chaque police en mois au 30/06/2026 (date d'arrêt), et regrouper les polices par tranche d'ancienneté (moins d'1 an, 1 à 3 ans, plus de 3 ans). Comparer le ratio sinistres/primes par tranche.
5. Pour chaque police résiliée, déterminer si elle a eu au moins un sinistre **après** sa date de résiliation apparente (à partir des données disponibles) — discuter ce que cela peut révéler sur la qualité des données sources.
6. Écrire le résultat consolidé (une ligne par police, avec ses indicateurs agrégés de sinistralité) en Parquet, partitionné par **country**.

5 Pour aller plus loin

- Comparer le plan d'exécution (`explain()`) de la jointure du Niveau 2 avant et après un `broadcast()` sur `policies`. Ce dataset est-il assez petit pour que ce soit pertinent ?
- Réécrire une des agrégations du Niveau 2 en Spark SQL pur, et comparer la lisibilité avec la version DataFrame API.
- Mesurer l'effet d'un `repartition()` par `country` avant l'écriture Parquet du Niveau 3 sur le nombre de fichiers de sortie.
- Ajouter une colonne technique `ingested_at` et une notion simple de traçabilité de la source, comme point de départ d'une couche Bronze/Silver.